EEET2601 Engineering Computing 1

Control Statements

Quang Tran

RMIT UNIVERSITY

Expression, statements, and blocks

has a value after evaluate (int or float)

- An **expression** is a formula consists of operators, variables, constants, array element, and even function calls
- A **statement** is a command to ask the computer to take an action, such as collect input, display to the screen, or do a calculation
- Excepting control statements, a statement ends by a semicolon ";"
- Braces { and } are used to group declarations and statements together into one **block** (or **compound statement**). Example:

```
int main(void) {

    …

}
```

Control statements

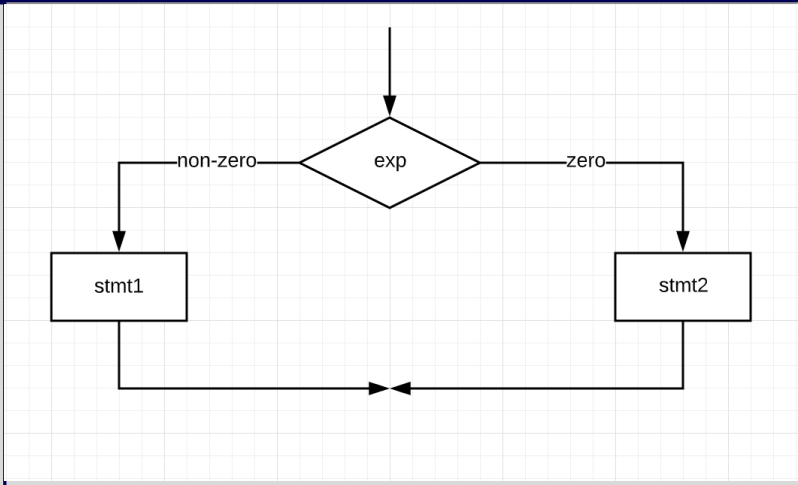
C provides two types of control statements to give you more control:

- **Decision** (or **conditional**): to perform different statements depending on a condition
  - Example: find out a given integer is even or odd
- **Loop**: to perform some statements repeatedly until a terminated condition is met
  - Example: print “Hello World” 1,000,000 times

Decision: if-else

```
if (exp) {
    stmt1;
} else {
    stmt2;
}
```

- stmt1 runs when exp is **non-zero** (for TRUE)
- stmt2 runs when exp is **zero** (for FALSE)
- else clause is optional



Example: if-else

```c
#include <stdio.h>

int main(void) {
    printf("Is there an overcast now? [Y/N]: ");
    char ans = getchar();

    if (ans == 'Y') {
        printf("Don't forget to bring your raincoat when you leave home\n");
    } else {
        printf("Just bring your hat when you leave home\n");
    }

    return 0;
}
```

Example: nested if-else

```c
#include <stdio.h>

int main(void) {
    // Declare variables to store two integers
    int num1, num2;

    // Get two integers in one line
    printf("Enter two integers: ");
    scanf("%d %d", &num1, &num2);

    // Compare two integers
    if (num1 > num2) {
        printf("The first number is greater than the second number\n");
    } else {
        if (num2 > num1) {
            printf("The second number is greater than the first number\n");
        } else {
            printf("Two numbers are equal to each other\n");
        }
    }

    return 0;
}
```

Decision: else-if

```
if (exp1) {
    stmt1;
} else if (exp2) {
    stmt2;
} else if (exp3) {
    stmt3;
} else if (…) {
    …
} else {
    stmt;
}
```

Example: else-if

```c
#include <stdio.h>

int main(void) {
    // Declare variables to store two integers
    int num1, num2;

    // Get two integers in one line
    printf("Enter two integers: ");
    scanf("%d %d", &num1, &num2);

    // Compare two integers
    if (num1 > num2) {
        printf("The first number is greater than the second number\n");
    } else if (num2 > num1) {
        printf("The second number is greater than the first number\n");
    } else {
        printf("Two numbers are equal to each other\n");
    }

    return 0;
}
```

Decision: switch

```
switch (exp) {
    case value1:
        stmt1;
        break;
    case value2:
        stmt2;
        break;
    …
    default:
        stmt;
}
```

float
↑
can compare large & smaller but not equal

- `exp`, `value1`, `value2` must be **integer** or **char** values
- the `break` statement terminates the switch statement
- the `break` statement is optional but without it, all cases presented after the matched case will always execute
- `default` is optional. It executes when `exp` doesn't match any case values

Example: switch

```c
#include <stdio.h>

int main(int argc, char const *argv[]) {
    printf("Enter a character: ");
    char ch = getchar();
    switch (ch) {
        case 'A':
            printf("You entered character 'A'\n");
            break;
        case 'B':
            printf("You entered character 'B'\n");
            break;
        case 'C':
            printf("You entered character 'C'\n");
            break;
        default:
            printf("The character you entered is not 'A', 'B', or 'C'\n");
    }

    return 0;
}
```

Conditional operator

- Syntax: `condition? value1 : value2;`
- Return `value1` if `condition` is non-zero and `value2` otherwise

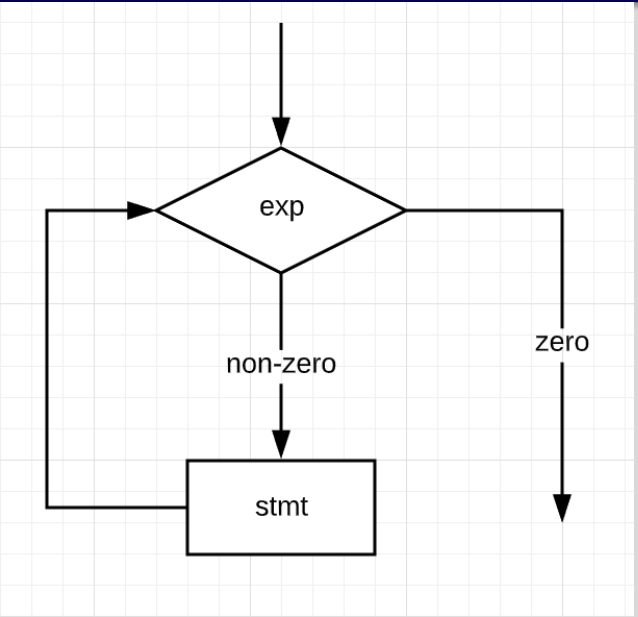
```
1  if (a > b) {    // 5 lines
2      max = a;
3  } else {
4      max = b;
5  }
```

```
max = (a > b)? a : b;    // 1 line
```

Loop: while

```
while (exp) {
    statement;
}
```

- while a pre-test loop
- What will happen if exp is actually 1 (or any non-zero numbers)?



Example: while

```c
#include <stdio.h>

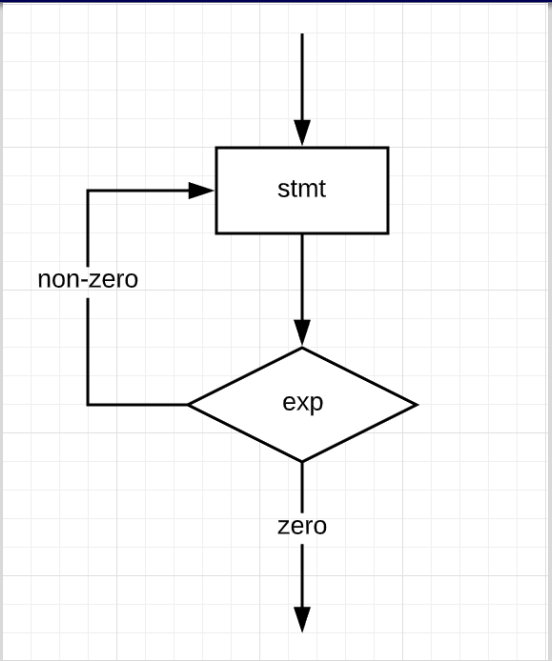
int main(void) {
    int i = 0;
    while (i < 5) {
        printf("%d\n", i);
        i++;
    }

    return 0;
}
```

Loop: do-while

```
do {
    statement;
} while (exp);
```

- do-while is a post-test loop



Example: do-while

```c
#include <stdio.h>

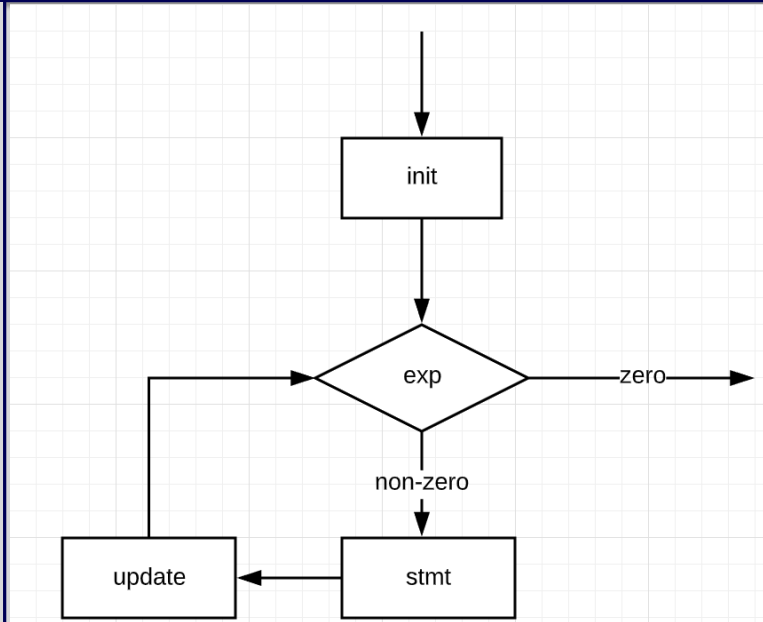
int main(void) {
    int i = 0;
    do {
        printf("%d\n", i);
        i++;
    } while (i < 5);

    return 0;
}
```

Loop: for

```
for (init; exp; update) {
    statement;
}
```

- for is a pre-test loop
- **init:** initialize a start value
- **exp:** expression using the value
- **update:** update the value
- init, exp, update are optional and can be omitted



Example: for

```c
#include <stdio.h>

int main(void) {
    for (int i = 0; i < 5; i++) {
        printf("%d\n", i);
    }

    return 0;
}
```

break

- Recall that the **break** statement was used earlier to terminate a switch
- It can also be used to terminate a loop

```c
#include <stdio.h>

int main(void) {
    for (int i = 0; i < 10; i++) {
        if (i == 5) break;
        printf("%d\n", i);
    }

    return 0;
}
```

continue

- The **continue** statement skip the remaining code of the current iteration and move to the next iteration

```c
#include <stdio.h>

int main(void) {
    for (int i = 0; i < 10; i++) {
        if (i == 5) continue;
        printf("%d\n", i);
    }

    return 0;
}
```

References

1. Fresh2Refresh, C Programming Tutorial, 2020.
2. StudyTonight, C Programming Language Tutorial, 2020.
3. LucidChart, Flowchart Tutorial, 2020.
4. Jumail bin Taliba, Introduction to Flowcharting, 2020.